

Cash Flow Minimizer — Quiz 2 Report

Course: Design and Analysis of Algorithms (DAA)
Student: Naufal Dariskarim — 5025231027

1. Program Overview

Cash Flow Minimizer is a web-based application that solves the debt-settlement problem among a group of people. Given a set of arbitrary transactions (who owes whom, and how much), the program computes the **minimum number of transactions** required to settle all debts completely.

This is achieved using a **Greedy Algorithm** that works by repeatedly matching the person with the highest debt against the person with the highest credit, reducing the total number of required payments.

2. Program Functionality

Feature	Description
Add Member	Add individuals to the group
Remove Member	Remove a member and all associated transactions
Record Transaction	Input a payment record: who paid for whom, and the amount
Delete Transaction	Remove a specific transaction from history
Simplify Debts	Automatically computes the minimal set of settlements using the Greedy algorithm

Core Algorithm — Greedy Cash Flow Minimization

- Calculate Net Balance** — For each person, compute `total received - total paid`. A positive balance means they are owed money (creditor); a negative balance means they owe money (debtor).
- Separate Creditors and Debtors** — Split all members into two sorted lists based on their net balance (descending order).
- Greedy Match** — At each step, pair the largest debtor with the largest creditor:
 - Transfer `min(debt, credit)` between them.
 - Reduce both balances accordingly.
 - Move to the next person if their balance reaches zero.
- Repeat** until all balances are settled.

Time Complexity: $O(N \log N)$ — dominated by the sorting step.

3. Program Flow



State Flow (React)



The result is derived state — it recalculates automatically whenever members or transactions change, with no manual trigger needed.

4. Technologies Used

Technology	Role
React 19	UI framework — component-based rendering and state management

Vite 8	Build tool and local development server with Hot Module Replacement (HMR)
JavaScript (ES2024)	Core programming language
JSX	HTML-in-JS syntax extension for React components
Lucide React	Icon library (Wallet, Receipt, RefreshCw, etc.)
CSS-in-JS (inline styles)	Component styling via JavaScript style objects
Node.js + npm	Package manager and runtime environment

No external CSS frameworks or backend services are used. The entire algorithm runs client-side in the browser.

5. How to Run Locally

Prerequisites

Make sure you have the following installed:

- [Node.js \(https://nodejs.org/\)](https://nodejs.org/) (v18 or higher)
- npm (comes bundled with Node.js)

Steps

1. Navigate to the project directory

```
cd "Q2_DAA"
```

2. Install dependencies

```
npm install
```

3. Start the development server

```
npm run dev
```

4. Open in browser

The terminal will display:

```
VITE ready in ~200ms
→ Local: http://localhost:5173/
```

Open <http://localhost:5173/> in your browser.

Stopping the Server

Press **Ctrl + C** in the terminal to stop the dev server.

Build for Production (optional)

```
npm run build
```

Output will be in the `dist/` folder, ready for static hosting.

6. Project Structure

```
Q2_DAA/
├── public/           # Static assets
├── src/
│   ├── App.jsx      # Main component – all logic and UI
│   ├── App.css      # Additional styles
│   ├── main.jsx     # React entry point
│   └── index.css     # Global CSS reset
├── index.html       # HTML shell
├── package.json     # Project metadata and dependencies
├── vite.config.js   # Vite configuration
└── Report.md        # This file
```